Tenchu Z Recompiled

Bringing the Xbox 360 game to PC and Mac as native code

Aulcreen · October-November 2026 · github.com/Aulcreen/TenchuZ-Xbox-Recomp

This document and the project contain no game data. You need your own copy of Tenchu Z to play. Tenchu Z belongs to its owners; this project isn't affiliated with FromSoftware, K2, Microsoft or Valve.

Overview

Tenchu Z came out on the Xbox 360 in 2007 and never got a PC release. Emulators can run it, but with the usual costs: a heavy CPU and GPU load, 30 fps, and graphics bugs on some cards. I wanted the game to run as a real PC program instead.

So I took the game's executable, translated its PowerPC code into C++ and compiled that for Windows and macOS. No CPU emulator is involved. On Windows I then replaced the emulated Xbox 360 GPU with my own Direct3D 12 renderer, which draws each frame directly. With that in place I added what a PC version should have: 60 fps, higher resolutions, anti-aliasing, keyboard and mouse, online play over Steam, and an installer.

Two parts are still emulated: the Xbox's XMA audio decoder, and the GPU on Mac.

Status

Every function of the game's own code runs as compiled PC code; nothing is interpreted. What remains emulated is part of the hardware the game expects. Counting the four parts it relies on equally, my estimate is **about 85% native on Windows** and **about 60% on macOS**.

Part	Windows	macOS	Why
Game code (CPU)	100%	100%	All of it recompiled to native code
Xbox kernel and dashboard	100%	100%	Reimplemented in native C++
Graphics (GPU)	95%	0%	Windows: my renderer; a small shim still answers the game's fences and register writes. Mac: GPU emulated on Vulkan
Audio	50%	50%	The game's XACT audio code is native; the XMA decoder chip is emulated
Overall	**~85%**	**~60%**	Average of the four parts

These are estimates by component, not a measurement of running time.

1. Recompilation

Everything starts from a verified disc image. The game's `default.xex` was built in April 2007 with XDK 2.0.2920, and it carries Microsoft's Direct3D, XACT audio, WMV movie and C runtime libraries inside it. That matters: those libraries get recompiled with the game, which is what later made the native renderer possible.

Step	What I did
Extraction	Wrote a disc extractor (`xiso.py`) and an XEX tool that decrypts and decompresses the executable into a raw PowerPC image I could study.
Recompilation	Used the ReXGlue SDK, which grew out of Xenia and XenonRecomp, to translate every function to C++. About 5.6 MB of PowerPC code turns into a 32 MB x86-64 program, built with Clang.
Missing functions	The code generator missed a few: four functions without unwind data, the game's `setjmp`/`longjmp` (now native), and 385 functions that are only ever reached through pointers. I wrote a tool that finds those from vtables and the instructions that build their addresses.
Xbox system calls	The kernel and dashboard (XAM) run as native C++ in the SDK. Tenchu needed a few fixes on top: pressing Start crashed on unset profile settings, starting a mission went black, and saves needed to live next to the game.

Step	What I did
Crash reports	A crash handler logs the game's registers and a named backtrace, so a player's report points straight at the function.

On the first day the game reached its title screen, the menus, character creation and the training dojo.

Part of the system	On an emulator	In this port
Game code, incl. D3D, XACT, WMV	PowerPC, emulated	Native x86-64 (ARM64 on Mac)
Xbox kernel and XAM	Emulated	Native C++
GPU on Windows	Xenos command stream emulated	Native Direct3D 12 renderer
GPU on macOS	-	Xenos emulation on Vulkan (MoltenVK)
Audio decoding	XMA hardware emulated	Still emulated
Memory layout	Big-endian	Big-endian, as recompiled code requires

2. Renderer

Emulating the Xbox 360 GPU means decoding its command stream and modelling its EDRAM, and that's where most of an emulator's graphics cost and bugs come from. Since the game's Direct3D library is recompiled code like everything else, I could hook it and draw the frame myself. Sonic Unleashed Recompiled and the Dante's Inferno port showed the approach works.

I mapped the library first, with 1,101 trace hooks across drawing, state, textures, shaders, render targets, clears, resolves, tiling and presentation. Then I wrote a decoder that rebuilds the GPU registers at every draw, and learned the device's memory layout from 5,464 gameplay draws. The game's shaders are translated to DXBC with the SDK's translator, and in play none fail.

Getting the picture right took longer than getting it on screen. The first native frames were darker than the emulator's, with red and blue swapped. Resolves were ignoring their destination format, and the game's display gamma ramp wasn't applied. After those fixes native output matched the emulator to within 2/255 on average. The renderer became the default on 20 October.

Fixes

Bug	Cause
Flickering	The primitive reset index, plus several menu, loading and mission cases.
GPU hang entering a mission	A texture was freed while the GPU was still using it.
Characters turning black with blood	The game clears to transparent black; I had reused the last clear's opaque black.
Missing shadows	Tiled render passes were resolved at the wrong moment.

3. NVIDIA

Players on NVIDIA cards saw black and coloured speckles over characters' faces, arms, legs and belt ties, and over tatami floors. The same thing shows up in Xenia Canary on NVIDIA. My own card is AMD and looked fine, but Windows' software renderer (WARP) showed the speckles too, so I could reproduce it.

Switching off the alpha test made them disappear, and MSAA, early depth and alpha-to-coverage had nothing to do with it. Some of the character shaders keep only fully opaque texels, testing alpha against 1.0 or 254/255. A texel stored as 255 filters to exactly 1.0 on AMD and on the Xbox 360's ATI GPU, but to a fraction less on NVIDIA. Pixels that should pass failed at random.

The fix compares alpha at the 8-bit precision the textures were made in: the reference moves half a step in the forgiving direction. NVIDIA and WARP now look like AMD, and the AMD image didn't change.

4. Framerate

The 30 fps limit turned out to be the game asking Direct3D to present every second vblank. Gameplay already scales with frame time, so I set it to every vblank in 3D scenes and checked 30 and 60 side by side: movement, jumping, attacks, the stealth meter, the camera and the mission movies all match.

The menus are different. They move a fixed amount per frame, so at 60 fps every fade ran twice as fast. Frames without a 3D scene, and the pause menu, stay at 30 fps. The wind and one HUD pulse also update at 30 Hz inside 60 fps frames. For smooth pacing, the game times its vblanks from the display, and the CPU and GPU work on consecutive frames at once.

I tried frame rates above 60 and took them out again: some blood effects advance per frame and hung in the air at 120 fps.

5. Performance

None of this changes the game's look; it only makes the image sharper and smoother. The internal resolution goes up to 4x of 720p and is chosen automatically from how long the GPU takes per frame. There's 4x MSAA, FXAA, contrast-adaptive sharpening and anisotropic filtering, all switchable while playing.

Most of the performance work was about doing less. The figures below were measured on a Radeon RX 6600; the later ones at 4x resolution to keep the GPU the bottleneck.

Change	Result
Cheaper draws, CPU and GPU frames overlapped (village, uncapped)	75 to 185 fps
Depth tested before shading where a draw allows it	+2%
Shadow map drawn without MSAA	0.7 ms a frame at 1440p
Tiled resolves done in hardware; compressed depth read directly	+12%
Whole render targets handed over instead of copied	+2.3%
Compiled pipelines kept between sessions	fewer first-time stutters

6. Controls

Keyboard and mouse work with rebindable keys and buttons, and the camera follows the mouse directly rather than imitating a stick. Xbox controllers work as they are, and other controllers through Steam Input. Steam's overlay hides controllers from the game, so I read them through Steam's own input API. Esc opens the settings menu.

7. Online

Tenchu Z's System Link is a LAN mode. I gave each player a virtual network address and carry the game's traffic over Steam's peer-to-peer connections, so the game believes it's on a LAN. It runs under Spacewar (App ID 480), Valve's public test app, which is why Steam shows players as playing "Spacewar". There's no Tenchu Z server; you connect straight to your friends. Friends can join through lobbies or the invite list in the Esc menu.

The first online match between two machines, a Windows PC and a Mac, worked on 24 October. Xbox Live modes aren't covered.

8. Blood

I wanted a bit more realism here, and admittedly a bit more gore. I mapped the game's effect pools and its character event handler to change how blood behaves. Blood on your clothes now stays until you swim, instead of fading after half a minute, and a bloody stealth kill stains you as well. Blood on the ground stays for the whole level, and when a character has taken more than 64 splats the older ones dry into a darker layer underneath. Turning it off in the Esc menu restores the original behaviour.

9. macOS

The Mac build runs natively on Apple Silicon. The disk image has the app and a Set Up Game Files step that copies the game from your own disc image. It draws through the SDK's GPU emulation on Vulkan, since my renderer is Direct3D 12 only. Controllers, including Bluetooth ones, and Steam online play work, and Mac and Windows players can play together.

10. Release

The Windows installer checks your disc image before it installs anything (it hashes `default.xex`), copies the game data and creates shortcuts. You don't need the image afterwards, and there's an uninstaller. Achievements unlock as on the Xbox and are saved on the PC.

Version 0.1.0 was released on 4 November 2026 for Windows and macOS. The code is under GPL-3.0. Valve's Steamworks SDK isn't in the repository; only the library Valve allows developers to ship comes with the game.

11. Roadmap

Audio still goes through the emulated XMA decoder, and that's a choice. The decoding itself already runs on the PC through FFmpeg; only the Xbox decoder chip's interface is emulated. Replacing it would be a lot of work for no gain in sound or speed I could expect, and converting the sounds at install time would grow them from 42 MB to 333 MB in memory, more than the game's 512 MB allows. The Mac still emulates the GPU. Widescreen beyond 16:9, a Linux build and Xbox Live modes aren't there yet, and I still want to play through every stage to catch what's left.

Timeline

2026	Milestone
18 Oct	Project started; the game reaches the title, menus and training dojo
19 Oct	Native renderer: shaders, draws, textures, colour matching
20 Oct	Native renderer by default; higher resolution, MSAA, FXAA; System Link
21 Oct	Installer, settings menu, keyboard and mouse, 60 fps; black blood fixed
24 Oct	macOS build; Steam controllers; first Windows-Mac online match
28 Oct	Filtering and sharpening, new installer, shadows fixed, public lobbies
29 Oct	Lasting blood, automatic resolution, single-sample shadow map
30 Oct	NVIDIA speckles fixed; resolves 12% faster
31 Oct	Render targets handed over instead of copied
4 Nov	Public repository and version 0.1.0

Credits

The ReXGlue SDK by Tom Clay, with code from the Xenia project by Ben Vanik and contributors (BSD 3-Clause), made this possible. Sonic Unleashed Recompiled and the ReXGlue ports of Dante's Inferno and Ace Combat 6 showed the way. Steamworks is by Valve. Tenchu and Tenchu Z belong to their owners.